[Hyder Abbas]

Spring Framework: Spring Boot

Learning Java Development is About: Learning the following tech

A. Java Language (Core Java)

B. Java Technology (JDBC, Servlet, JSP, JSTL, etc.)

- **Java Database Connectivity (JDBC):** A set of APIs that allow Java programs to interact with databases.
- **Servlets:** Java programs that run on a web server to handle HTTP requests and responses.
- **JSP (Java Server Pages):** Used to create dynamic web content by embedding Java code in HTML.

C. Framework (Hibernate, Spring, Spring Boot, Microservices, REST APIs, etc.)

- **A framework is not a new technology but an abstraction built on top of existing technologies to simplify development.**

- **A framework provides a set of pre-built functionalities that help you build applications more easily.**

- **It reduces the need to write repetitive boilerplate code by offering reusable components and automated tasks, such as object creation, configuration, or database management.**

Spring:

- **Spring is a comprehensive framework for building enterprise-level applications.**

- **Spring manages objects for you. Based on the configuration provided, it creates objects, injects dependencies, and performs object lifecycle management.**

Telusko

[Hyder Abbas]

Important Key Features of Spring:

1. Easier Java Development:

- Spring provides a comprehensive set of tools and best practices that make it easier to develop Java applications.
- It simplifies the development process by managing infrastructure concerns (like transactions, security, and database access), allowing developers to focus more on business logic.

2. End-to-End Backend Development:

- Spring can be used to develop the complete backend of an application, from handling user requests in the web layer (Spring MVC) to managing data access (Spring Data), and handling business logic in service layers.
- It is flexible enough to build monolithic applications or microservices-based architectures.

3. Versatility:

- Spring integrates easily with other technologies and frameworks, making it a versatile choice for modern application development.
- For example, it integrates with **Hibernate** and **JPA** for data access, enabling seamless database operations while abstracting complexity.

4. Object Management:

- Spring manages how **objects are created** and **used**, reducing the complexity of your code by handling object lifecycle and dependency management.
- With **Dependency Injection (DI)**, Spring helps in decoupling components, making the codebase more modular and maintainable.

5. Dependency Injection (DI):

- Spring allows you to **inject dependencies** (pass objects into other objects), which makes your code **flexible**, **loosely coupled**, and **easier to test**.
- DI promotes better separation of concerns and easier unit testing of individual components.

6. Modular Structure:

- Spring is organized into a wide array of **modules** (e.g., **Spring MVC** for web applications, **Spring Data** for data access, **Spring Security** for authentication and authorization, etc.).

Telusko

Spring makes your code more testable, maintainable, and loosely coupled.

Whether you're building a monolithic application or microservices, Spring provides the tools and flexibility to support a wide range of applications with ease.

Spring Core:

- **The Spring Core module is the foundation of all other modules and provides fundamental features such as Dependency Injection, Inversion of Control (IoC), and Bean management.**

- **Dependency Management:** The Spring Core module supplies the Spring container to perform Dependency Management through Inversion of Control (IoC). This helps in decoupling components

Dependency Management in Spring

Dependency Management in Spring is the process of assigning **dependent objects** to a **target object** by loading both classes and creating their instances. This is typically done using **Dependency Injection** (DI), where one class (the dependent class) is injected into another (the target class) to manage the relationships between objects.

1. **Target Class**:
    - The class that depends on the services or functionality provided by another class is called the **target class**.
2. **Dependent Class**:
    - The class that provides the necessary services or functionality to the target class is called the **dependent class**.

Common Modes of Dependency Injection

Setter Injection

- Setter Injection is the process of injecting dependencies into a target class through setter methods.
- The Spring container first creates an instance of the target class, and then calls the appropriate setter method to inject the dependent object.

Teluskо

Constructor Injection

- Constructor Injection involves passing the dependent object to the target object via the constructor.
- This ensures that all required dependencies are provided at the time of object creation, making the object immutable and fully initialized.

Field Injection

- Field Injection involves directly injecting dependencies into the fields of the target class using reflection.
- The Spring container automatically assigns the dependency to the field at runtime.

Spring Containers (IoC Containers)

The Spring Core module provides two types of IoC containers:

1. **BeanFactory Container**
2. **ApplicationContext Container** (Latest and most widely used)

Summary of Differences:

Feature	BeanFactory	ApplicationContext
Initialization Type	Lazy Initialization (beans are created when needed)	Eager Initialization (beans are created at startup)
Advanced Features	Basic container, lacks advanced features	Provides additional features like event propagation, and annotation support

Teluskо

Performance	Faster startup with memory efficiency	Slower startup, but better runtime performance and availability of all beans
Example	`XmlBeanFactory` (deprecated in favor of `ApplicationContext`)	`AnnotationConfigApplicationContext`, `ClassPathXmlApplicationContext`

For most Spring applications today, `ApplicationContext` is recommended due to its richer feature set and better alignment with modern development practices.

Spring Application Development Approaches

Spring applications can be developed using the following approaches:

1. **XML Configuration (Legacy Approach):**
   - Traditional configuration approach where Spring beans and dependencies are configured in an XML file.
   - Mainly used in legacy or maintenance projects.
2. **Annotation-Driven Configuration:**
   - A mix of XML and Java configuration using annotations like `@Component`, `@Autowired`, `@Bean`, etc.
   - Easier to manage and more flexible than XML alone.
3. **Java-Based Configuration:**
   - Configuration is entirely done using Java code (using `@Configuration`, `@Bean` annotations) and does not require XML configuration files.
   - The most modern and widely used approach in Spring development.
4. **Spring Boot Auto-Configuration:**
   - Supported by Spring Boot, this approach simplifies configuration by automatically configuring Spring beans based on the application's requirements.
   - Spring Boot automatically detects the necessary dependencies and applies the correct configuration, making the process seamless.

Teluisko

Spring Boot:

Spring Boot is one of the most efficient and simplified approaches to working with the Spring Framework, offering a streamlined way to build Spring-based applications with minimal configuration.

Spring Boot = Spring + Auto Configuration + Embedded Servers + Actuators

Spring Boot is a combination of:

- **Spring** (Core framework)
- **XML configurations** (reduced or eliminated)
- **Auto Configuration** (automatic setup based on context)
- **Embedded Servers** (no need to configure an external server)
- **Actuators** (for monitoring and management)

Key Features of Spring Boot:

1. Simplified Application Development:

Spring Boot simplifies the development of Spring-based applications by removing the need for complex configurations. Developers can focus more on business logic and less on infrastructure setup.

2. Types of Applications You Can Build with Spring Boot:

Spring Boot allows developers to create various types of applications easily:

- **Standalone Applications**: Create simple, self-contained Java applications that run on their own without requiring an external web server.
- **Web Applications**: Build web applications using embedded servers like Tomcat, Jetty, or Netty.
- **Distributed Applications**: Develop distributed systems or web services, including RESTful APIs, to interact with other applications.

Advantages of Using Spring Boot:

1. Starter POMs (Simplified Maven/Gradle Configuration):

Teusko

Spring Boot provides a set of pre-configured "starter" dependencies that simplify your Maven or Gradle build configuration. These starters automatically configure common components, allowing you to focus on writing your business logic.

- **web-starter**: For creating web applications.
- **datajpa-starter**: For integrating Spring Data JPA.
- **security-starter**: For enabling Spring Security.
- **mail-starter**: For integrating email functionality.

2. Auto Configuration:

Spring Boot takes care of most of the configuration automatically based on the context of the application.

- **Database Connection Pooling**: Spring Boot automatically configures database connections, removing the need for manual setup.
- **Embedded Server Deployment**: Spring Boot can deploy the web application in an embedded server, eliminating the need for an external web server like Tomcat.
- **IOC Container Management**: The Spring Boot application will start the **Inversion of Control (IoC)** container and manage your beans automatically.
- **Component Scanning**: Spring Boot will scan for components, services, and repositories based on the configuration, reducing the need for manual bean registration.

3. Actuators:

Spring Boot includes **Actuators** that help you monitor and manage your application. These are essential for maintaining application health and performance.

- **Bean Count**: See how many beans are loaded in your application.
- **URL Mappings**: View all URL patterns mapped in the application.
- **Loaded Configuration Properties**: List all properties that are loaded in the application.
- **Application Health**: Check the health status of your application, including custom health checks.
- **Heap Dump**: Get a snapshot of the JVM memory.
- **Thread Dump**: Inspect the application threads for performance issues.

4. Embedded Servers:

Spring Boot comes with built-in support for various embedded web servers, eliminating the need for deploying your application to an external server.

- **Apache Tomcat (default)**: Spring Boot comes pre-configured with Tomcat as the default embedded server.
- **Jetty**: Another lightweight and fast server option.
- **Netty**: A non-blocking I/O server designed for high-performance applications.

Tejusko

Creating a Spring Boot Application:

1. Initializr Website (start.spring.io):

You can create a Spring Boot application by visiting Spring Initializr, where you can generate a custom Spring Boot project with your desired dependencies and configuration.

2. IDE (Integrated Development Environment):

Many IDEs, such as **IntelliJ IDEA** and **Eclipse , STS**, provide direct integration with the Spring Initializr website, allowing you to create and configure Spring Boot applications seamlessly.

> **Note:** IDEs internally connect to **start.spring.io** to generate the Spring Boot project (Spring Starter Project).

Core Annotations in Spring Framework

Spring provides a wide range of annotations that simplify configuration, improve flexibility, and enhance the ease of use for developers. Below is an overview of the core annotations used in Spring, categorized into **stereotype annotations**, **dependency injection annotations**, and others.

What is a Spring Bean?

A **Spring Bean** is any Java class (whether predefined, user-defined, or from a third-party library) whose object is created, managed, and maintained by the **Spring container**. Spring Beans are at the heart of Spring's IoC mechanism and undergo lifecycle management, including dependency injection.

What Does `@SpringBootApplication` Do?

The `@SpringBootApplication` annotation is a convenience annotation that combines three important annotations:

1. **`@SpringBootConfiguration`**: Equivalent to `@Configuration` in Spring, it marks the class as the configuration class for the Spring Boot application.
2. **`@EnableAutoConfiguration`**: Tells Spring Boot to automatically configure the application based on the dependencies present in the classpath.

Telusko

3. `@ComponentScan`: Enables Spring to scan for components, configurations, and services in the package and sub-packages of the class.

Stereotype Annotations:

These annotations help in defining the roles or responsibilities of the beans within the Spring container. They are used to mark a class as a special type of Spring bean.

`@Component`

- **Purpose**: Marks a class as a Spring bean, indicating that it will be auto-detected through classpath scanning and instantiated in the Spring container.
- **Usage**: Use `@Component` when you want to create a generic Spring bean.

`@Service`

- **Purpose**: A specialization of `@Component`, used to annotate service-layer beans. It provides better readability and indicates that the class is holding business logic.
- **Usage**: Use `@Service` to mark service classes that contain business logic or service-related operations.

`@Repository`

- **Purpose**: A specialization of `@Component`, used to annotate DAO (Data Access Object) classes. It marks the class as a data access object (typically responsible for interacting with the database).
- **Usage**: Use `@Repository` to annotate classes responsible for database operations.

`@Controller`

- **Purpose**: A specialization of `@Component`, used to mark classes that handle HTTP requests in Spring MVC applications. It is typically used for Spring Web MVC controller classes.
- **Usage**: Use `@Controller` to define a Spring MVC controller class.

`@RestController`

- **Purpose**: A combination of `@Controller` and `@ResponseBody`. It is used in RESTful web service development and automatically serializes response data to JSON or XML.
- **Usage**: Use `@RestController` for creating REST APIs in Spring MVC.

Telusko

`@Configuration`

- **Purpose**: Indicates that a class is a source of bean definitions. This annotation is used for classes that define bean methods annotated with `@Bean`.
- **Usage**: Use `@Configuration` when you want to declare beans programmatically in Java (instead of XML).

Dependency Injection Annotations:

These annotations are used to inject dependencies into Spring beans, supporting Spring's Inversion of Control (IoC) container.

`@Autowired`

- **Purpose**: Automatically injects the dependency into a Spring bean, either by constructor, setter method, or field. Spring uses the `@Autowired` annotation to wire dependencies automatically.
- **Usage**: Use `@Autowired` to inject dependencies into Spring beans.

`@Qualifier`

- **Purpose**: This annotation is used in conjunction with `@Autowired` when multiple beans of the same type are available. It helps in specifying which bean should be injected when there are multiple candidates.
- **Usage**: Use `@Qualifier` to specify the exact bean to inject.

`@Primary`

- **Purpose**: Specifies that a bean is the default choice for autowiring when multiple candidates of the same type are present.
- **Usage**: Use `@Primary` to mark a preferred bean when multiple beans are available for autowiring.

Bean Definition Annotations:

`@Bean`

- **Purpose**: Indicates that a method produces a bean that should be managed by the Spring container.
- **Usage**: Use `@Bean` within a class annotated with `@Configuration` to define a bean.

TeluSko

`@PostConstruct` and `@PreDestroy`

- **Purpose:** These annotations are used for lifecycle management. `@PostConstruct` is executed after the bean's initialization, and `@PreDestroy` is executed before the bean is destroyed.
- **Usage:** Use `@PostConstruct` to define logic that needs to run after the bean's creation, and `@PreDestroy` to define cleanup logic.

`@Profile`

- **Purpose:** Specifies that a bean is only available in certain environments or profiles (e.g., "development", "production").
- **Usage**: Use `@Profile` to define beans that should be loaded only in specific profiles.

[Hyder Abbas]

Telusko

Spring Data JPA = Spring ORM + Auto Implementation of Repositories + Query Methods + Custom Queries

Spring Data JPA is a part of the Spring Framework that simplifies the development of the Persistence Layer by reducing boilerplate code and auto-implementing repository methods based on interfaces.

What is Spring Data JPA?

Spring Data JPA is a library that:

- **Provides abstraction over the JPA (Java Persistence API).**
- **Automatically generates standard CRUD methods based on method names.**
- **Uses Hibernate as the default JPA provider internally.**
- **Simplifies interaction with the database using simple interfaces.**

Why Do We Need Spring Data JPA?

In traditional DAO development:
● For 1 table ⇒ we create 1 DAO class with 4 methods (insert, update, delete, select).
● For 5,000 tables ⇒ 5,000 DAO classes × 4 methods = 20,000 redundant methods.

👉 This is not scalable or maintainable.

✅ Spring Data JPA eliminates this by generating these methods dynamically through interface definitions.

Key Features of Spring Data JPA

1. **Auto Implementation of Repositories:**
   No need to write method logic — Spring creates the implementation at runtime.

2. **Derived Query Methods:**
   Define methods like `findByName()`, `findByPriceGreaterThan()`, and Spring will generate SQL automatically.

3. **Custom Queries Using @Query:**
   Write your own HQL or native SQL using annotations.

4. **Pagination and Sorting Support:**
   Built-in support through `JpaRepository`.

5. **Query By Example (QBE):**
   Dynamic querying using example objects.

Important Interfaces in Spring Data JPA

● `CrudRepository<T, ID>`

- **Basic CRUD operations**

TeUSKO

- **Methods:** `save()`, `findById()`, `deleteById()`, **etc.**

- `JpaRepository<T, ID>`
  - **Inherits from CrudRepository**
  - **Adds: Pagination, Sorting, Batch processing, etc.**

Spring Data JPA Application Setup

1. **Add Dependencies:**
   - `spring-boot-starter-data-jpa`
   - `mysql-connector-java`
   - `lombok`

2. **Configure Datasource in** `application.properties`**:**

```
spring.datasource.url=jdbc:mysql://localhost:3306/dbname
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

3. **Create Entity Class:**

```
@Entity
@Data
public class Book {
    @Id
    private Integer bookId;
    private String bookName;
    private Double bookPrice;
```

Teusko

```
}
```

4. **Create Repository Interface:**

```
public interface BookRepository extends JpaRepository<Book,
Integer> {}
```

5. **Use the Repository in Service or Controller.**

Common Methods in CrudRepository

- `save(entity)` – Insert or Update (Upsert)
- `saveAll(entities)` – Bulk insert/update
- `findById(id)` – Fetch record by primary key
- `findAll()` – Retrieve all records
- `deleteById(id)` – Delete by primary key
- `delete(entity)` – Delete by object
- `existsById(id)` – Check if record exists
- `count()` – Count total records

Derived Query Methods (findBy Methods)

- Spring generates query based on method name
- Supports keywords like `And`, `Or`, `Between`, `LessThan`, `GreaterThan`, `Like`, etc.

Examples:

- `findByBookName(String name)`
- `findByBookPriceGreaterThan(Double price)`

Teluisko

- `findByBookNameAndBookPrice(String name, Double price)`

Custom Queries Using @Query

If method names are too complex or if you want custom logic:

```
@Query("SELECT b FROM Book b WHERE b.bookPrice > :price")
List<Book> getBooksCostlierThan(@Param("price") Double price);
```

✔ You can use:
- **JPQL (HQL) – Entity and field-based**
- **Native SQL – Table and column-based (with `nativeQuery=true`)**

HQL vs SQL Queries

Aspect	HQL (JPQL)	SQL (Native)
Syntax	Uses entity and field names	Uses table and column names
DB Dependency	Independent	Dependent
Execution	Needs translation by Dialect	Directly executed by DB
Performance	Slightly slower than SQL	Generally faster
Maintenance	Easier and less error-prone	Less readable in large queries

---

Pagination Support

`JpaRepository` **provides built-in methods for pagination:**

Telusko

```
Page<Book> findAll(Pageable pageable);
```

- Use `PageRequest.of(page, size)` to pass page and size.

Query By Example (QBE)

Used for dynamic querying without writing query strings.

```
Book probe = new Book();
probe.setBookName("Spring");

Example<Book> example = Example.of(probe);
bookRepository.findAll(example);
```

Spring Profiles for Environment Specific Configs

Use profile-specific config files like:

- `application-dev.properties`
- `application-test.properties`
- `application-prod.properties`

Activate profile via:

properties

```
spring.profiles.active=dev
```

- **Spring will load properties based on the active environment.**

Telusko

✅ **Summary**

Spring Data JPA simplifies database operations by:
- **Eliminating boilerplate CRUD code**
- **Supporting dynamic query creation**
- **Providing powerful tools like pagination, sorting, and custom queries**
- **Easily integrating with Spring Boot**

==========

Spring WebMVC Concepts

==========

1) Spring MVC Architecture

- **Based on Front Controller Design Pattern**
- **DispatcherServlet as central servlet**

2) Request Handling Flow

- **Request → DispatcherServlet → HandlerMapping → Controller**
- **Controller → Service Layer → ViewResolver**

3) Core Annotations in Spring MVC

- `@Controller` **for Web Layer**
- `@RequestMapping`, `@GetMapping`, `@PostMapping` **etc.**

Telusko

- **`@PathVariable` and `@RequestParam` for reading data from URL**
- **`@ModelAttribute` for form binding**
- **`@RequestBody` and `@ResponseBody` for JSON/XML**

4) HandlerMethodReturn Types

- **Direct data (`String`, `Object`)**
- **Views (`ModelAndView`)**
- **Response Wrappers (`ResponseEntity`)**

5) Form Handling in Spring MVC

- **Data binding using form models**
- **Spring's built-in form tag library**
- **Validation with JSR-303 (Bean Validation)**

6) Exception Handling

- **Localized error handling using `@ExceptionHandler`**
- **Global error handling using `@ControllerAdvice`**

7) View Technologies Integration

- **JSP, Thymeleaf, or other template engines**
- **ViewResolver configuration**

8) File Upload Support

Telusko

- **MultipartResolver support**
- **Handling file uploads with `MultipartFile`**

9) Spring MVC + REST Integration

- **Using Spring MVC to expose REST APIs**
- **Returning JSON/XML from controller methods**
- **RESTful conventions (stateless communication)**

RESTful Services

==========

1) What is a Distributed Application?

- **Applications spread across multiple systems**
- **Communicate over the network**

2) Distributed Technologies

- **CORBA**
- **RMI**
- **EJB**
- **SOAP Web Services**
- **RESTful Services**

Teluskо

3) REST Architecture

- **Provider (Service Provider)**
- **Consumer (Service Consumer)**

4) HTTP Protocol

- **Request and Response model**
- **HTTP Methods (GET, POST, PUT, DELETE)**
- **HTTP Status Codes (1xx to 5xx categories)**

5) XML and JAX-B API

- **Used for structured data exchange**
- **XML data binding**
- **Marshalling (Java → XML)**
- **Unmarshalling (XML → Java)**

6) JSON and Jackson/Gson API

- **JSON as universal data format**
- **Data represented in key-value format**
- **Conversion between Java and JSON using third-party libraries**

7) Provider Development

- **Development of RESTful endpoints**
- **Use of annotations for HTTP method mapping**

Telusko

- **Reading data via request parameters or body**
- **Handling request and response media types**

8) Content-Type and Accept Headers

- **Used to define the data format in request and response**
- **Ensures proper communication between client and server**

9) Provider Testing using Postman

- **REST APIs can be tested manually using Postman tool**
- **Helps verify request/response structure and status codes**

10) Provider Documentation using Swagger

- **REST API documentation with Swagger UI**
- **Interactive testing interface for endpoints**

11) Consumer Development (Sync & Async)

- **REST API consumption using**
    - **RestTemplate (Synchronous - older)**
        - **WebClient (Synchronous & Asynchronous - modern)**

12) Exception Handling in REST API

- **Centralized exception handling mechanism**
- **Custom exception mapping using annotations**

Teluskо

Spring Boot Actuators

==========

✅ **Purpose:**

- **Provide production-ready monitoring and management endpoints for Spring Boot apps.**

✅ **Key Features:**

- **Exposes REST endpoints like `/actuator/health`, `/actuator/info`, etc.**
- **Used for observability, diagnostics, and runtime insights.**

✅ **Common Endpoints:**

- `/actuator/health` **→ Shows application health status.**
- `/actuator/info` **→ Custom application metadata.**
- `/actuator/metrics` **→ Application and system metrics (JVM, threads, GC, etc.)**
- `/actuator/env` **→ Application environment properties.**
- `/actuator/mappings` **→ Lists all request mappings.**
- `/actuator/beans` **→ Shows all Spring-managed beans.**
- `/actuator/threaddump` **and** `/actuator/heapdump` **→ Useful for debugging.**

✅ **Interview Points:**

Telusko

- **Useful in DevOps and monitoring tools like Prometheus, Grafana, etc.**
- **Actuators help implement health checks in microservices architecture.**
- **Can be secured and selectively exposed using Spring Security.**

==========

Spring HATEOAS (Hypermedia as the Engine of Application State)

==========

✅ **Purpose:**

- **Enhances REST API responses by including hypermedia links.**
- **Follows HATEOAS constraint of REST → makes API self-navigable.**

✅ **Key Concepts:**

- **Adds links (URIs) to responses to guide the client on next possible actions.**
- **Improves decoupling between client and server.**
- **Useful for building discoverable REST APIs.**

✅ **Benefits:**

- **Enables dynamic client interaction without hardcoding URLs.**

Telusko

- **Encouraged in REST maturity levels (Richardson Maturity Model Level 3).**

✅ **Interview Points:**

- **Part of RESTful best practices, though not always used in modern microservices.**
- **Makes APIs more robust and client-friendly.**

Hyder Abbas

==========

Swagger / OpenAPI Integration

==========

✅ **Purpose:**

- **Auto-generates REST API documentation.**
- **Provides interactive UI (Swagger UI) for API testing.**

✅ **Key Features:**

- **Documents endpoints, request/response models, HTTP methods.**
- **OpenAPI 3.0 is the current standard supported.**
- **Can be integrated using libraries like Springfox or springdoc-openapi.**

✅ **Benefits:**

Telusko

- **Enhances developer experience.**
- **Facilitates contract-first development.**
- **Helps in API onboarding and collaboration.**

✅ **Interview Points:**

- **Important when working with external teams or API-first companies.**
- **Used in API versioning, validation, and third-party testing.**

==========

Spring Bean Scopes

==========

✅ **Supported Scopes in Spring:**

Scope	Description
Singleton	One instance per Spring container (default)
Prototype	New instance each time it is requested
Request	One instance per HTTP request (web apps only)
Session	One instance per HTTP session (web apps only)
Application	One instance per ServletContext

TeluSko

==========

Global Exception Handling

==========

✅ **Purpose:**

- **Centralized mechanism to handle application-wide errors.**

✅ **Key Components:**

- `@ControllerAdvice` **→ Global exception handler class.**
- `@ExceptionHandler` **→ Maps specific exceptions to custom responses.**
- **Can handle both custom and standard exceptions.**
- **Integrates well with** `ResponseEntity` **for consistent error response format.**

✅ **Benefits:**

- **Avoids repetitive try-catch blocks in controllers.**
- **Provides clean, reusable error handling strategy.**
- **Can return structured error objects (status, message, timestamp).**

✅ **Interview Points:**

- **Frequently asked in error handling or API reliability questions.**

Telusko

- **Important for robust and user-friendly APIs.**

==========

Spring Boot – AOP (Aspect-Oriented Programming)

==========

✅ **Purpose:**

- **Provides a way to separate cross-cutting concerns (like logging, security, transactions) from business logic.**

✅ **Key Concepts:**

Term	Description
Aspect	**Class containing cross-cutting logic**
Advice	**Action taken at a join point (before, after, around)**
Join Point	**Specific point in execution (like method call)**
Pointcut	**Expression to match join points (e.g., method pattern)**
Weaving	**Linking aspects with main code**

✅ **Types of Advice:**

Telusko

- `@Before` → Runs before method execution
- `@After` → Runs after method execution
- `@AfterReturning` → Runs after method successfully returns
- `@AfterThrowing` → Runs if method throws exception
- `@Around` → Surrounds the method execution

==========

Spring Security – JWT & OAuth 2.0

==========

✅ JWT (JSON Web Token)

- Stateless, token-based authentication mechanism.
- Encodes user info in a token passed in HTTP Header (`Authorization: Bearer <token>`).

✅ Key JWT Terms:

- Header → Algorithm and token type
- Payload → Claims (user info)
- Signature → Ensures token integrity

✅ Benefits:

- No session needed (stateless)

Telusko

- **Scalable in distributed microservices**
- **Easy to integrate with frontend (Angular/React)**

✅ **OAuth 2.0**

- **Open standard for delegated access.**
- **Commonly used for third-party login (Google, GitHub, Facebook).**
- **Defines four roles:**
    - **Resource Owner (User)**
    - **Client (e.g., frontend app)**
    - **Authorization Server (Google, GitHub)**
    - **Resource Server (API that protects data)**

==========

General Interview Notes

==========

✅ **REST API Best Practices:**

- **Use nouns in URLs, not verbs.**
- **Use appropriate HTTP methods (GET, POST, PUT, DELETE).**
- **Follow status codes strictly (200, 201, 400, 404, 500).**

Telusko

- **Version APIs (e.g., `/api/v1/resource`).**
- **Enable pagination, sorting, filtering.**

✅ **Spring Boot Essentials:**

- **Auto-configuration and embedded servers.**
- **Uses starter dependencies to reduce boilerplate.**
- **Profiles for environment-specific configurations.**
- **Supports externalized configuration (`application.yml` or `.properties`).**

✅ **Dependency Injection (DI):**

- **Core principle of Spring Framework.**
- **Enables loose coupling and testability.**
- **Achieved via @Autowired, constructor injection, etc.**

✅ **Bean Scopes:**

- **Singleton (default)**
- **Prototype**
- **Request**
- **Session**
- **Application**

✅ **Component Scanning:**

Teluskо

- Scans for classes annotated with `@Component`, `@Service`, `@Repository`, `@Controller`.

✅ Spring Annotations Summary:

Annotation	Purpose
`@RestController`	Controller for REST APIs
`@Service`	Business layer logic
`@Repository`	Data access layer
`@Component`	Generic Spring-managed bean
`@Autowired`	Dependency injection
`@RequestMapping`	URL mapping
`@GetMapping`	HTTP GET method
`@PostMapping`	HTTP POST method
`@RequestBody`	Binds request payload
`@PathVariable`	Extracts values from path parameters
`@RequestParam`	Extracts values from query parameters
`@ResponseEntity`	Custom HTTP status + response body

[Hyder Abbas]

Teusko

Microservices Architecture

- **Definition: Breaking an application into independent, self-contained services.**
- **Each service: Own logic + database + can be deployed independently.**
- **Communication: Typically via REST APIs (HTTP) or messaging queues (Kafka, RabbitMQ).**
- **Benefits:**
    - **Scalability**
    - **Fault Isolation**
    - **Faster Deployment Cycles**
    - **Technology Agnostic**

☁️ Spring Cloud Components

🧭 1. Eureka Server (Service Registry)

- **Acts as a discovery server.**
- **Microservices register themselves (Eureka Clients).**
- **Clients can look up other service locations dynamically.**
- **Annotations:**
    - `@EnableEurekaServer` **(on registry)**

Teluskо

- `@EnableEurekaClient` or `@EnableDiscoveryClient` (on microservices)

- **Default Port: 8761**
- **Dashboard: `http://localhost:8761`**

🗒️ 2. Spring Cloud Config Server

- **Centralized configuration management.**
- **Externalizes application properties to Git/File system.**
- **Supports multiple environments (`dev`, `prod`, etc.)**

- **Annotation:**
  - `@EnableConfigServer`

URL to access config:

`http://localhost:8888/{application-name}/{profile}`

-

📦 3. Feign Client (Declarative REST Client)

- **Simplifies REST API calls between services.**
- **Load-balanced by default (integrates with Ribbon/LoadBalancer).**

Teluskо

- No manual `RestTemplate` code required.
- Annotation:
    - `@FeignClient(name = "service-name")`
- Enable Feign: `@EnableFeignClients`

🚪 4. Spring Cloud Gateway (API Gateway)

- Handles routing, authentication, load balancing, rate limiting.
- Replaces Zuul 1.x (lighter, reactive, Spring WebFlux-based).
- Central entry point to all microservices.
- Supports filters (pre/post routing).
- Properties-based Route Config or Java DSL (RouteLocator).

💥 5. Circuit Breaker Pattern (Resilience4J)

- Protects microservices from cascading failures.
- Circuit opens if downstream service is slow/unavailable.
- Alternative libraries: Resilience4j, Hystrix (deprecated).
- Integrated with:
    - Spring Boot
    - Feign

Teusko

    - **WebClient**

- **Annotations:**
    - `@CircuitBreaker(name = "serviceName", fallbackMethod = "fallbackMethod")`

- **Monitors health metrics via Actuator.**

🕵️ 6. Zipkin Server (Distributed Tracing)

- **Collects and visualizes traces across microservices.**
- **Helps identify latency issues.**

Telusko